

SQL

1.SQL概述

1.1 定义

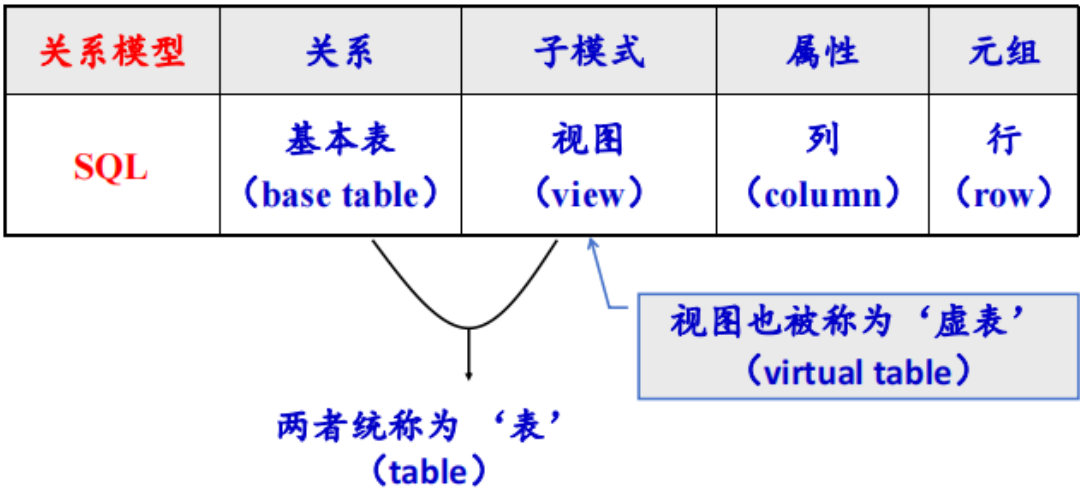
SQL是**结构化查询语言**，是关系数据库的标准语言

1.2 特点

- 1. 综合统一：
 - 集**数据定义语言(DDL)**、**数据操纵语言 (DML)**、**数据控制语言(DCL)**于一体
 - 可以独立完成数据库中的全部活动
 - 用户数据库投入运行后，可根据需要随时逐步修改模式，不影响数据库的运行
 - 数据操作符统一
- 2. 高度非过程化：只需提出做什么，存取路径的选择以及操作过程由系统自动完成
- 3. 面向集合的操作方式
- 4. 以同一种语法结构提供多种使用方式：
 - SQL是独立的语言，能够独立地用于联机交互的使用方式
 - SQL又是嵌入式语言，能够嵌入到高级语言中
- 5. 语言简洁，易学易用

1.3 SQL语言的基本语言成分

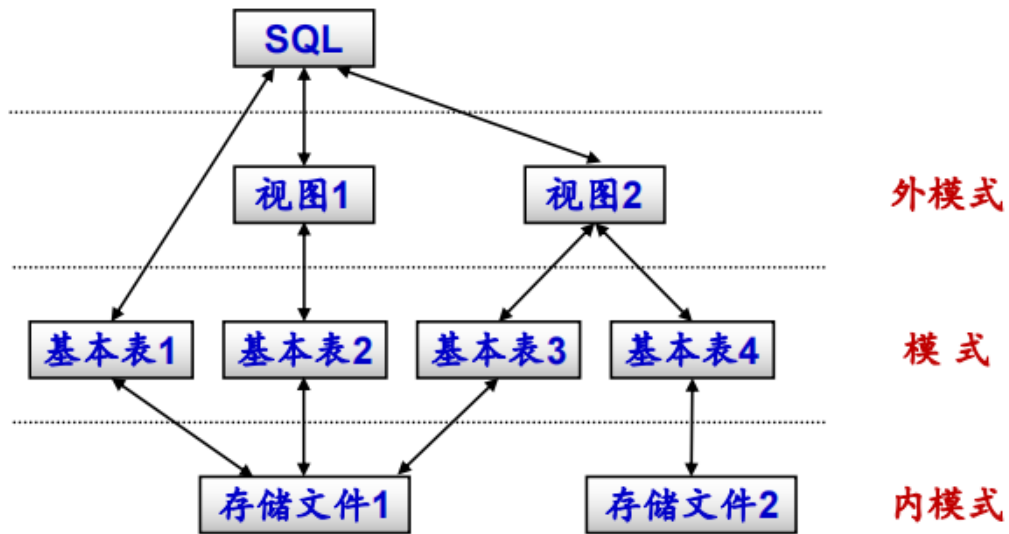
□基本概念的变化



- 符号
- 保留字:在SQL语言中具有特定含义的语言成分，通常是一个英文单词，用于标识一条SQL语句的组成成分，也包括系统内置的系统表、存储过程/函数、SQL函数等
- 标识符：用于对用户创建的数据库对象进行命名，包括表、视图、属性、存储过程/函数、触发器、变量
- 常量：数值常量、字符常量、日期/时间常量

- SQL对大小写不敏感
- 数值类型的常量不需要定界符，但字符（串）或日期/时间类型的常量需要使用‘单引号’作为定界符，可支持不同的日期显示格式

1.4 SQL与关系数据库的三级模式



基本表

- 本身独立存在的表
- 一个关系对应一个基本表
- 一个（或多个）基本表对应一个存储文件
- 一个表可以带若干索引，索引也放在存储文件中

存储文件

- 逻辑结构组成了关系数据库的内模式
- 物理结构对最终用户是隐蔽的

视图

- 从一个或几个基本表导出的表
- 数据库中只存放视图的定义而不存放视图对应的数据，这些数据仍存在导出视图的基本表中
- 本身不独立存储在数据库中，视图是一个虚表
- 视图在概念上与基本表等同，用户可以在视图上再定义视图

视图的优点：

- 视图能够简化用户的操作
- 视图使用户能以多种角度看待同一数据
- 视图对重构数据库提供了一定程度的逻辑独立性
- 视图能够对机密数据提供安全保护

哪些视图是可以更新的？

基本表的行列子集视图一般是可更新的。

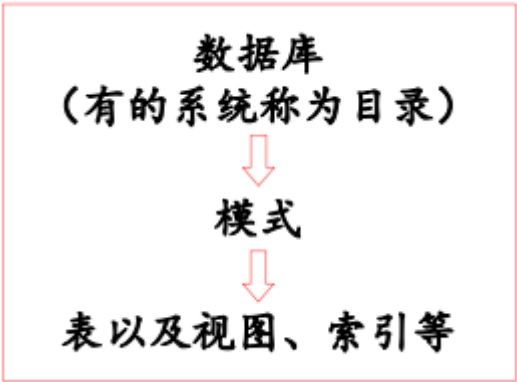
若视图的属性来自聚集函数、表达式，则该视图肯定是不可更新的。

2.学生-课程数据库

3.SQL数据定义

3.1 层次化的数据库对象命名机制

- 一个关系数据库管理系统的实例中可以建立多个数据库
- 一个数据库中建立多个模式
- 一个模式下通常包括多个表、视图和索引



3.2 数据定义

表 3.3 SQL 的数据定义语句

操作对象	操作方式		
	创建	删除	修改
模式	CREATE SCHEMA	DROP SCHEMA	
表	CREATE TABLE	DROP TABLE	ALTER TABLE
视图	CREATE VIEW	DROP VIEW	
索引	CREATE INDEX	DROP INDEX	ALTER INDEX

SQL不提供修改模式和修改视图定义的操作，用户如果想修改这些对象，只能先将它们删除再重建。

3.2.1 模式定义与删除

定义模式实际上定义了一个命名空间。在这个空间中可以定义该模式包含的数据库对象，例如基本表、视图、索引等。

```
1 CREATE SCHEMA<模式名> AUTHORIZATION<用户名>
2
3 1.
4 #为用户WANG定义一个学生-课程模式S_T
5 CREATE SCHEMA "S_T" AUTHORIZATION WANG;
6
7 2.
8 CREATE SCHEMA AUTHORIZATION WANG;
9 #该语句没有指定<模式名>，<模式名>隐含为<用户名>
10
11 3.
12 #为用户ZHANG创建了一个模式TEST，并且在其中定义一个表TAB1
```

```

13 CREATE SCHEMA TEST AUTHORIZATION ZHANG
14 CREATE TABLE TAB1 (COL1 SMALLINT,
15                     COL2 INT,
16                     COL3 CHAR(20),
17                     COL4 NUMERIC(10,3),
18                     COL5 DECIMAL(5,2));

```

删除模式：

```

1 DROP SCHEMA <模式名> <CASCADE|RESTRICT>
2
3 CASCADE（级联）：
4 #删除模式的同时把该模式中所有的数据库对象全部删除
5 RESTRICT（限制）：
6 #如果该模式中定义了下属的数据库对象（如表、视图等），则拒绝该删除语句的执行。仅当该模式中没有
  任何下属的对象时才能执行。
7
8 #例：
9 DROP SCHEMA ZHANG CASCADE;
10 #删除模式ZHANG；同时该模式中定义的表TAB1也被删除。

```

3.2.2 定义基本表

建表的同时通常还可以定义与该表有关的完整性约束条件。这些约束条件被存入系统的数据字典中。

模式与基本表：每一个基本表都属于某一个模式，一个模式包含多个基本表。

定义基本表时定义所属模式的方法：

- 在表名中显示给出，用模式.表的形式
- 在创建模式语句中同时也创建表
- 设置所属的模式，这样在创建表时表名不必给出模式名

当用户创建基本表时若没有指定模式，系统根据搜索路径来确定该对象所属的模式。搜索路径的默认值为\$user,PUBLIC。

```

1 CREATE TABLE <表名>
2   (<列名> <数据类型>[ <列级完整性约束条件> ]
3   [, <列名> <数据类型>[ <列级完整性约束条件>] ]
4   ...
5   [, <表级完整性约束条件> ] );
6
7 #<表名>：所要定义的基本表的名字
8 #<列名>：组成该表的各个属性（列）
9 #<列级完整性约束条件>：涉及相应属性列的完整性约束条件
10 #<表级完整性约束条件>：涉及一个或多个属性列的完整性约束条件
11 #如果完整性约束条件涉及到该表的多个属性列，则必须定义在表级上，否则既可以定义在列级也可以定义在表级

```

示例：

□[例3.5] 建立“学生”表Student。学号是主码，姓名取值唯一。

```
CREATE TABLE Student
```

```
(Sno CHAR(9) PRIMARY KEY,
```

主码

```
/* 列级完整性约束条件,Sno是主码*/
```

```
Sname CHAR(20) UNIQUE,
```

```
/* Sname取唯一值*/
```

```
Ssex CHAR(2),
```

```
Sage SMALLINT,
```

```
Sdept CHAR(20)
```

UNIQUE
约束

```
);
```

□[例3.7] 建立一个学生选课表SC

```
CREATE TABLE SC (
```

```
Sno CHAR(9),
```

```
Cno CHAR(4),
```

```
Grade SMALLINT,
```

```
PRIMARY KEY (Sno,Cno),
```

```
/* 主码由两个属性构成，必须作为表级完整性进行定义*/
```

```
FOREIGN KEY (Sno) REFERENCES Student(Sno),
```

```
/* 表级完整性约束条件，Sno是外码，被参照表是Student */
```

```
FOREIGN KEY (Cno)REFERENCES Course(Cno)
```

```
/* 表级完整性约束条件， Cno是外码，被参照表是Course*/
```

```
);
```

参照表和被参照表可以是同一个表。

3.2.3 数据类型

SQL中域的概念用数据类型表示；定义表的属性时需要指明数据类型及其长度。

3.2.4 修改基本表

```
1 ALTER TABLE <表名>
2 [ ADD[COLUMN] <新列名> <数据类型> [ 完整性约束 ] ]
3 [ ADD <表级完整性约束>]
4 [ DROP [ COLUMN ] <列名> [CASCADE| RESTRICT] ]
5 [ DROP CONSTRAINT<完整性约束名>[ RESTRICT | CASCADE ] ]
6 [ALTER COLUMN <列名><数据类型> ] ;
7
8 #<表名>是要修改的基本表
9 #ADD子句用于增加新列、新的列级完整性约束条件和新的表级完整性约束条件
10 #DROP COLUMN子句用于删除表中的列：
11 #如果指定了CASCADE短语，则自动删除引用了该列的其他对象
12 #如果指定了RESTRICT短语，则如果该列被其他对象引用，关系数据库管理系统将拒绝删除该列
13 #DROP CONSTRAINT子句用于删除指定的完整性约束条件
14 #ALTER COLUMN子句用于修改原有的列定义，包括修改列名和数据类型
```

示例：

[例3.8] 向Student表增加“入学时间”列，其数据类型为日期型

```
ALTER TABLE Student ADD S_entrance DATE;
```

不管基本表中原来是否已有数据，新增加的列一律为空值

[例3.9] 将年龄的数据类型由字符型（假设原来的数据类型是字符型）改为整数。

```
ALTER TABLE Student ALTER COLUMN Sage INT;
```

[例3.10] 增加课程名称必须取唯一值的约束条件。

```
ALTER TABLE Course ADD UNIQUE(Cname);
```

3.2.5 删除基本表

```
1 DROP TABLE <表名> [RESTRICT | CASCADE] ;
2
3 #RESTRICT: 删除表是有限制的:
4 #欲删除的基本表不能被其他表的约束所引用
5 #如果存在依赖该表的对象，则此表不能被删除
6 #默认情况是RESTRICT
7 #CASCADE: 删除该表没有限制:
8 #在删除基本表的同时，相关的依赖对象一起删除
```

示例：

❑ **[例3.11]** 删除Student表

```
DROP TABLE Student CASCADE;
```

➢ 基本表定义被删除，数据被删除

➢ 表上建立的索引、视图、触发器等一般也将被删除

❑ **[例3.12]** 若表上建有视图，选择RESTRICT时表不能删除；选择CASCADE时可以删除表，视图也自动删除。

```
CREATE VIEW IS_Student
```

```
AS
```

```
SELECT Sno, Sname, Sage
```

```
FROM Student
```

```
WHERE Sdept='IS';
```

```
DROP TABLE Student RESTRICT;
```

—ERROR: cannot drop table Student because other objects depend on it

```
DROP TABLE Student CASCADE;
```

—NOTICE: drop cascades to view IS_Student

3.2.6 索引

建立索引目的：加快查询速度

常见索引：

- 顺序文件上的索引
- B+树索引

- 散列索引
- 位图索引：使用位向量

索引虽然能够加速数据库查询，但需要**占用一定的存储空间**，当基本表更新时，索引要进行相应的维护。

关系数据库管理系统在执行查询时会自动选择合适的索引作为存取路径，用户不必也不能显式地选择索引。索引是关系数据库管理系统的内部实现技术，属于内模式的范畴。

建立索引：

```

1  #语句格式：
2  CREATE [UNIQUE] [CLUSTER] INDEX <索引名>
3  ON <表名>(<列名>[<次序>][, <列名>[<次序>] ]...);
4
5  #<表名>：要建索引的基本表的名字
6  #索引：可以建立在该表的一列或多列上，各列名之间用逗号分隔
7  #<次序>：指定索引值的排列次序，升序：ASC，降序：DESC。缺省值：ASC
8  #UNIQUE：此索引的每一个索引值只对应唯一的数据记录
9  #CLUSTER：表示要建立的索引是聚簇索引

```

□[例3.13] 为学生-课程数据库中的Student，Course，SC三个表建立索引。Student表按学号升序建唯一索引，Course表按课程号升序建唯一索引，SC表按学号升序和课程号降序建唯一索引

```

CREATE UNIQUE INDEX Stusno ON Student(Sno);

CREATE UNIQUE INDEX Coucno ON Course(Cno);

CREATE UNIQUE INDEX SCno ON SC(Sno ASC, Cno DESC);

```

修改/删除索引：

```

1  ALTER INDEX <旧索引名> RENAME TO <新索引名>
2  #例：将SC表的SCno索引名改为SCSno
3  ALTER INDEX SCno RENAME TO SCSno;
4
5  DROP INDEX <索引名>;
6  #删除索引时，系统会从数据字典中删去有关该索引的描述。
7  #例：删除Student表的Stusname索引
8  DROP INDEX Stusname;

```

3.2.7 数据字典

数据字典是关系数据库管理系统内部的一组系统表，记录了数据库中所有定义信息。

4.SQL数据查询

4.1 概述

映像语句：

目标子句	SELECT * colname { , colname ... }
范围子句	FROM tablename { , tablename ... }
条件子句	[WHERE search_condition]
分组子句	[GROUP BY colname { , colname ... }
分组查询子句	[HAVING group_condition]]
排序输出子句	[ORDER BY colname [ASC DESC] { , colname [ASC DESC] ... }] ;

4.1.1 目标子句

定义生成结果关系时目标属性的投影方式

4.1.2 范围子句

指定操作对象（被访问的关系）

可以在FROM子句中对一个关系起别名

SELECT子句和FROM子句是一条映像语句中必不可少的两个部分

4.1.3 条件子句

用于定义查询条件

包含单个关系中的元组选择条件以及关系与关系之间的连接条件

4.2 单表查询

只涉及一个表的查询。

4.2.1 数据查询语句一般格式

```
1  SELECT [ALL|DISTINCT] <目标列表表达式>[ ,<目标列表表达式>] ...
2  FROM <表名或视图名>[ ,<表名或视图名> ]...(SELECT 语句) [AS]<别名>
3  [ WHERE <条件表达式> ]
4  [GROUP BY <列名1> [ HAVING <条件表达式> ] ]
5  [ORDER BY <列名2> [ ASC|DESC ] ] ;
6
7  #SELECT子句：指定要显示的属性列
8  #DISTINCT：去掉表中重复的行
9  #如果没有指定DISTINCT关键字，则默认为ALL，即保留结果表中取值重复的行
10 #FROM子句：指定查询对象（基本表或视图）
11 #WHERE子句：指定查询条件
```


12	#GROUP BY 子句：对查询结果按指定列的值分组，该属性列值相等的元组为一个组。通常会在每组中作用聚集函数。
13	#HAVING 短语：只有满足指定条件的组才予以输出
14	#ORDER BY 子句：对查询结果表按指定列值的升序或降序排序

查询时可以使用列别名改变查询结果的列标题。

SELECT子句的<目标列表达式>不仅可以是表中的属性列，也可以是表达式。

4.2.2 查询满足条件的元组

查 询 条 件	谓 词
比 较	=, >, <, >=, <=, !=, <>, !>, !<
确定范围	BETWEEN ... AND ... , NOT BETWEEN ... AND ...
确定集合	IN, NOT IN
字符匹配	LIKE, NOT LIKE
空 值	IS NULL, IS NOT NULL
多重条件（逻辑运算）	AND, OR, NOT

字符匹配：

```

1  #谓词：
2  [ NOT ] LIKE '<匹配串>' [ ESCAPE '<换码字符>' ]
3  #<匹配串>可以是一个完整的字符串，也可以含有通配符%（任意长度（长度可以为0）的字符串）和 _
   （任意单个字符）
4  #例如：a%b表示以a开头，以b结尾的任意长度的字符串
5  #例如：a_b表示以a开头，以b结尾的长度为3的任意字符串
6
7  #1. 匹配串为固定字符串
8  #例：查询学号为201215121的学生的详细情况
9  SELECT * FROM Student WHERE Sno LIKE '201215121';
10 #等价于：
11 SELECT * FROM Student WHERE Sno = '201215121';
12
13 #2. 匹配串为含通配符的字符串
14 #例：查询所有姓刘学生的姓名、学号和性别。
15 SELECT Sname, Sno, Ssex
16 FROM Student
17 WHERE Sname LIKE '刘%';
18 #例：查询姓"欧阳"且全名为三个汉字的学生的姓名。
19 SELECT Sname
20 FROM Student
21 WHERE Sname LIKE '欧阳___';
22
23 #3. 使用换码字符将通配符转义为普通字符，转义作用
24 #ESCAPE '\ ' 表示“\”为换码字符
25 #例：查询DB_Design课程的课程号和学分
26 SELECT Cno, Ccredit

```

```
27 FROM Course
28 WHERE Cname LIKE 'DB\_Design' ESCAPE '\';
29 #例：查询以"DB_"开头，且倒数第3个字符为 i 的课程的具体情况
30 SELECT *
31 FROM Course
32 WHERE Cname LIKE 'DB\_%i\_ _' ESCAPE '\';
```

4.2.3 对查询结果排序

ORDER BY子句：

- 可以按一个或多个属性列排序
- 对于空值：排序时显示的次序由具体系统实现来决定
- 各个系统的实现可以不同，只要保持一致就行

示例：

□[例3. 39] 查询选修了3号课程的学生学号及其成绩，查询结果按分数降序排列。

```
SELECT Sno, Grade
FROM SC
WHERE Cno= '3'
ORDER BY Grade DESC;
```

□[例3. 40] 查询全体学生情况，查询结果按所在系的系号升序排列，同一系中的学生按年龄降序排列。

```
SELECT *
FROM Student
ORDER BY Sdept, Sage DESC;
```

4.2.4 聚集函数

作用：方便用户，增强检索功能。

当聚集函数遇到空值时，除了COUNT(*)外，都跳过空值而只处理非空值。

在一个空集上进行计算，COUNT返回0，其他的返回NULL

聚集函数	功能描述
COUNT(*)	统计元组个数
COUNT([DISTINCT ALL] <列名>)	统计一列中值的个数
SUM([DISTINCT ALL] <列名>)	计算一列值的总和（此列必须为数值型）
AVG([DISTINCT ALL] <列名>)	计算一列值的平均值（此列必须为数值型）
MAX([DISTINCT ALL] <列名>)	求一列值中的最大值
MIN([DISTINCT ALL] <列名>)	求一列值中的最小值

示例：

❑ [例3.41] 查询学生总人数。

```
SELECT COUNT(*)  
FROM Student;
```

❑ [例3.42] 查询选修了课程的学生人数。

```
SELECT COUNT(DISTINCT Sno)  
FROM SC;
```

❑ [例3.43] 计算1号课程的学生平均成绩。

```
SELECT AVG(Grade)  
FROM SC  
WHERE Cno = '1';
```

❑ [例3.44] 查询选修1号课程的学生最高分数。

```
SELECT MAX(Grade)  
FROM SC15  
WHERE Cno='1';
```

❑ [例3.45] 查询学生201215012选修课程的总学分数。

```
SELECT SUM(Ccredit)  
FROM SC, Course  
WHERE Sno='201215012' AND  
SC.Cno=Course.Cno;
```

4.2.5 对查询结果分组

GROUP BY子句分组：

GROUP BY：对数据进行分组

HAVING：对分组的数据在指定条件下取出

细化聚集函数的作用对象：

- 如果未对查询结果分组，聚集函数将作用域整个查询结果
- 对查询结果分组后，聚集函数将分别作用于每个组
- 按指定的一列或多列值分组，值相等的为一组

示例：

❑ [例3.46] 求各个课程号及相应的选课人数。

```
SELECT Cno, COUNT(Sno)  
FROM SC  
GROUP BY Cno;
```

❑ 查询结果可能为：

Cno	COUNT(Sno)
1	22
2	34
3	44
4	33
5	48

❑ [例3.47] 查询选修了3门以上课程的学生学号。

```
SELECT Sno  
FROM SC  
GROUP BY Sno  
HAVING COUNT(*) > 3;
```

❑ **HAVING**短语与**WHERE**子句区别：

- 作用对象不同
- **WHERE**子句作用于基表或视图，从中选择满足条件的元组
- **HAVING**短语作用于组，从中选择满足条件的组。

这里要注意WHERE子句是不能用聚集函数作为条件表达式的。

聚集函数只能用于SELECT子句和GROUP BY中的HAVING子句。

WHERE子句和HAVING子句的作用对象不同：

WHERE子句作用于基表或视图。

4.3 连接查询

4.3.1 定义

- 连接查询：同时涉及两个以上的表的查询
- 连接条件或连接谓词：用来连接两个表的条件
- 连接字段：连接谓词中的列名称

4.3.2 连接操作的执行过程

- 嵌套循环法(NESTED-LOOP)
- 索引连接(INDEX-JOIN)
- 排序合并法(SORT-MERGE)

4.3.3 自身连接

一个表与其自己进行连接

- 需要给表起别名区别
- 需要使用别名前缀

4.3.4 外连接

4.3.5 多表连接

4.4 嵌套查询

4.4.1 嵌套查询概述

一个SELECT-FROM-WHERE语句称为一个**查询块**。

将一个**查询块**嵌套在另一个**查询块**的WHERE子句或HAVING短语的条件中的查询称为**嵌套查询**。

上层的查询块称为**外层查询**或**父查询**。

下层查询块称为**内层查询**或**子查询**。

SQL语言允许多层嵌套查询，即一个子查询中还可以嵌套其他子查询。

子查询的限制：不能使用ORDER BY子句。

有些嵌套查询可以用连接运算替代：谨慎使用嵌套查询。

4.4.2 嵌套查询求解方法

- 不相关子查询：**子查询的查询条件不依赖于父查询**，独立子查询；执行过程：**由里向外，逐层处理**
- 相关子查询：**子查询的查询条件依赖于父查询**；执行过程：**由外向里，嵌套循环**
- 求解相关子查询不能像求解不相关子查询那样一次将子查询求解出来，然后求解父查询。内层查询由于与外层查询有关，因此必须反复求值。

4.4.3 含有IN谓词的子查询

示例:

□ [例3.55] 查询与“刘晨”在同一个系学习的学生。

□ 使用‘IN + 不相关子查询’

```
SELECT Sno, Sname, Sdept
FROM Student
WHERE Sdept IN (
    SELECT Sdept
    FROM Student
    WHERE Sname='刘晨');
```

□ 使用‘IN + 相关子查询’

```
SELECT x.Sno, x.Sname, x.Sdept
FROM Student x
WHERE '刘晨' IN (
    SELECT y.Sname
    FROM Student y
    WHERE y.Sdept=x.Sdept);
```

4.4.4 带有比较运算符的子查询

当能确切知道内层查询返回单值时，可用比较运算符>, <, =, >=, <=, !=或<>来代替IN谓词。

示例:

□ 在[例3.55]中，由于一个学生只可能在一个系学习，则可以用 = 代替IN：

```
SELECT Sno, Sname, Sdept
FROM Student
WHERE Sdept = (SELECT Sdept
                FROM Student
                WHERE Sname = '刘晨');
```

□ 说明：由于姓名Sname不是学生关系Student的码，并不能保证此子查询一定返回单值，所以要慎用属性与子查询之间的直接比较！

□ [例3.57] 找出每个学生超过他选修课程平均成绩的课程号。

```
SELECT Sno, Cno
FROM SC x
WHERE Grade >= ( SELECT AVG(Grade)
                  FROM SC y
                  WHERE y.Sno = x.Sno );
```

相关子查询

□ 说明：

- 在本例中，子查询肯定只返回一个统计结果（单值），可放心直接比较；
- 即使子查询仅返回单值，也不是所有数据库系统都支持这种类型的表示方式，建议使用IN或带有SOME、ANY、ALL量词的量化比较！

4.4.5 带有ANY (SOME) 或ALL谓词的子查询

□使用ANY或ALL谓词时必须同时使用比较运算 (SOME与ANY是同义词)

谓词	语义	谓词	语义
> ANY	大于子查询结果中的某个值	> ALL	大于子查询结果中的所有值
< ANY	小于子查询结果中的某个值	< ALL	小于子查询结果中的所有值
>= ANY	大于等于子查询结果中的某个值	>= ALL	大于等于子查询结果中的所有值
<= ANY	小于等于子查询结果中的某个值	<= ALL	小于等于子查询结果中的所有值
= ANY	等于子查询结果中的某个值	= ALL	等于子查询结果中的所有值 ()
!= ANY	不等于子查询结果中的某个值	!= ALL	不等于子查询结果中的任何一个值
<> ANY		<> ALL	

□‘= ALL’与‘!= ANY’通常没有实际使用价值。

示例:

带有 ANY (SOME) 或 ALL 谓词的子查询 2

□[例3.58]查询非计算机科学系中
比计算机科学系任意一个学生年
龄小的学生姓名和年龄

```
SELECT Sname,Sage
FROM Student
WHERE Sage < ANY (
    SELECT Sage
    FROM Student
    WHERE Sdept = 'CS')
AND Sdept <> 'CS';
/*父查询块中的条件 */
```

□结果:

Sname	Sage
王敏	18
张立	19

□执行过程:

- 首先处理子查询，找出CS系中所有学生的年龄，构成一个集合 { 20, 19 }
- 再处理父查询，找所有不是CS系且年龄 小于20 或 小于19 的学生

带有 ANY (SOME) 或 ALL 谓词的子查询 3

□用聚集函数实现[例 3.58]

```
SELECT Sname,Sage
FROM Student
WHERE Sage < (
    SELECT MAX (Sage)
    FROM Student
    WHERE Sdept= 'CS' )
AND Sdept <> 'CS';
```

等价关系:

	=	<> 或 !=	<	<=	>	>=
ANY	IN		< MAX	<= MAX	> MIN	>= MIN
ALL		NOT IN	< MIN	<= MIN	> MAX	>= MAX

4.4.6 带有EXISTS谓词的子查询

带有EXISTS谓词的子查询不返回任何数据，只产生逻辑真值。

由EXISTS引出的子查询，其目标列表表达式通常都是*，因为返回的是逻辑真值，与具体的列无关。

示例：

□ [例3. 60] 查询所有选修了1号课程的学生姓名。

□ 思路分析：

- 本查询涉及Student和SC关系
- 在Student中依次取每个元组的Sno值，用此值去检查SC表
- 若SC中存在这样的元组，其Sno值等于此Student. Sno值，并且其Cno='1'，则取此Student. Sname送入结果表

```
SELECT Sname
FROM Student S
WHERE EXISTS ( SELECT *
                FROM SC
                WHERE SC.Sno = S.Sno AND Cno = '1');
```

- 不同形式的查询之间的替换：

一些带EXISTS或NOT EXISTS谓词的子查询不能被其他形式的子查询等价替换。

所有带IN谓词、比较运算符、ANY和ALL谓词的子查询都能用带EXISTS谓词的子查询等价替换。

- 用EXISTS/NOT EXISTS实现全称量词

SQL中没有全称量词，可以把带有全称量词的谓词转换为等价的带有存在量词的谓词。

示例：

□ [例3. 55] 查询与“刘晨”在同一个系学习的学生。

□ IN + 子查询

```
SELECT Sno, Sname, Sdept
FROM Student
WHERE Sdept IN (
    SELECT Sdept
    FROM Student
    WHERE Sname = '刘晨');
```

可以用‘EXISTS+子查询’替换

```
SELECT Sno, Sname, Sdept
FROM Student S1
WHERE EXISTS (
    SELECT *
    FROM Student S2
    WHERE S2.Sdept=S1.Sdept
    AND S2.Sname = '刘晨');
```


这里会用到谓词逻辑的知识：

□[例3. 63]查询至少选修了学生201215122选修的全部课程的学生号码。

□解题思路：

➤查询语义的解析：如果学号为x的学生满足查询条件，那么对所有的课程y，只要201215122学生选修了课程y，则x也选修了y。

➤形式化表示：

- 用p表示谓词 “学生201215122选修了课程y”

- 用q表示谓词 “学生x选修了课程y”

- 则上述查询为： $(\forall y) p \rightarrow q$

- 等价变换：

$$\begin{aligned}(\forall y) p \rightarrow q &\equiv \neg (\exists y (\neg (p \rightarrow q))) \\ &\equiv \neg (\exists y (\neg (\neg p \vee q))) \\ &\equiv \neg \exists y (p \wedge \neg q)\end{aligned}$$

➤变换后语义如下：

不存在这样的课程y，学生201215122选修了y，而学生x没有选。

□用NOT EXISTS谓词表示如下：

```
SELECT DISTINCT Sno
FROM SC SCX
WHERE NOT EXISTS (
    SELECT *
    FROM SC SCY
    WHERE SCY.Sno='201215122' AND
        NOT EXISTS (
            SELECT *
            FROM SC SCZ
            WHERE SCZ.Sno=SCX.Sno AND
                SCZ.Cno=SCY.Cno ) );
```

4.5 集合查询

集合操作的分类：

- 并操作UNION
- 交操作INTERSECT
- 差操作EXCEPT

参加集合操作的各个子查询需要满足：

- 结果的列数必须相同
- 对应列的数据类型也必须相同

示例：

❑UNION：将多个查询结果合并起来时，系统自动去掉重复元组

❑UNION ALL：将多个查询结果合并起来时，保留重复元组

❑[例3.64] 查询计算机科学系的学生及年龄不大于19岁的学生。

```
SELECT *  
FROM Student  
WHERE Sdept = 'CS'  
UNION  
SELECT *  
FROM Student  
WHERE Sage <= 19;
```

❑[例3.65] 查询选修了课程1或者选修了课程2的学生。

```
SELECT Sno  
FROM SC  
WHERE Cno = '1'  
UNION  
SELECT Sno  
FROM SC  
WHERE Cno = '2';
```

4.6 基于派生表查询

子查询还可以出现在FROM子句中，这时子查询生成的临时派生表成为主查询的查询对象。

示例：

❑[例3.57] 找出每个学生超过他自己选修课程平均成绩的课程号

```
SELECT Sno, Cno  
FROM SC, (SELECT Sno, Avg(Grade)  
FROM SC  
GROUP BY Sno)  
AS Avg_sc(avg_sno, avg_grade)  
WHERE SC.Sno = Avg_sc.avg_sno  
and SC.Grade >= Avg_sc.avg_grade
```

如果子查询没有聚集函数，派生表可以不指定属性列，子查询SELECT子句后面的列名为其缺省属性。

示例：

❑[例3.60] 查询所有选修了1号课程的学生姓名

❑可以用如下查询完成：

```
SELECT Sname  
FROM Student,  
(SELECT Sno FROM SC WHERE Cno='1') AS SC1  
WHERE Student.Sno = SC1.Sno;
```

5.SQL数据更新

5.1 插入数据

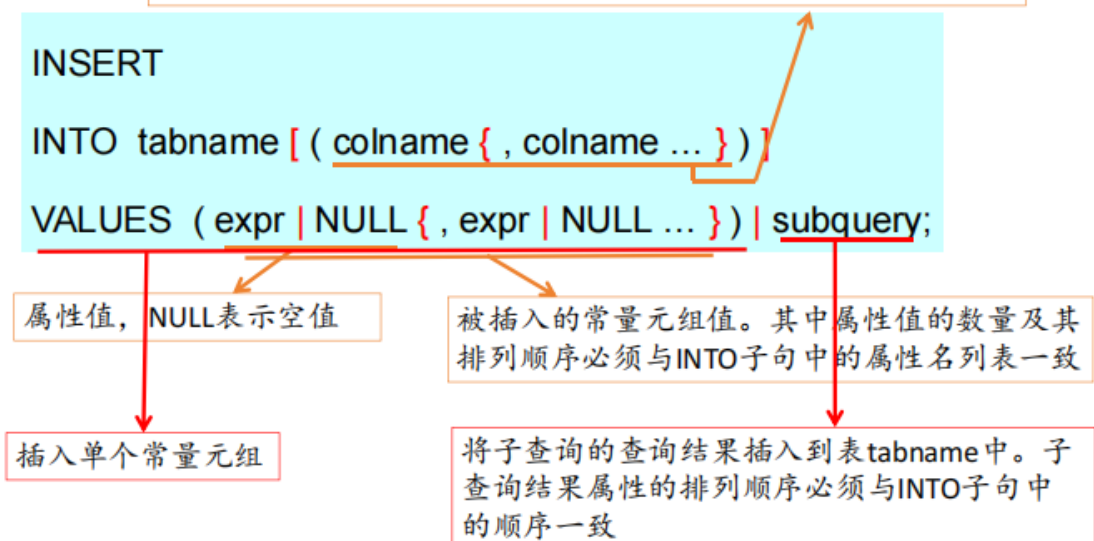
5.1.1 插入元组

```

1  #将新元组插入指定表中，语句格式：
2  INSERT
3  INTO <表名> [( <属性列1> [, <属性列2> ... ]) ]
4  VALUES ( <常量1> [, <常量2> ] ... );
5
6  INTO子句
7  #指定要插入数据的表名及属性列
8  #属性列的顺序可与表定义中的顺序不一致
9  #没有指定属性列：表示要插入的是一条完整的元组，且属性列属性与表定义中的顺序一致
10 #指定部分属性列：插入的元组在其余属性列上取空值
11
12 VALUES子句
13 #提供的值必须与INTO子句匹配：值的个数 & 值的类型

```

属性名列表可以被省略。在此情况下，属性的排列顺序采用基表定义中的顺序



□ [例3.69] 将一个新学生元组（学号：201215128；姓名：陈冬；性别：男；所在系：IS；年龄：18岁）插入到Student表中。

```

INSERT
INTO Student (Sno, Sname, Ssex, Sdept, Sage)
VALUES ('201215128', '陈冬', '男', 'IS', 18);

```

□ [例3.70] 将学生张成民的信息插入到Student表中。

```

INSERT
INTO Student
VALUES ('201215126', '张成民', '男', 18, 'CS');

```

5.1.2 插入子查询结果

```
1 #语句格式
2     INSERT
3     INTO <表名> [( <属性列1> [, <属性列2>... )]
4     子查询;
5
6 #子查询:
7 #SELECT子句目标列必须与INTO子句匹配
8 #值的个数 & 值的类型
```

[例3.72]对每一个系，求学生的平均年龄，并把结果存入数据库

□ **第一步：建表**

```
CREATE TABLE Dept_age (
    Sdept CHAR(15)          /* 系名 */
    Avg_age SMALLINT );    /* 学生平均年龄 */
```

□ **第二步：插入数据**

```
INSERT
INTO Dept_age(Sdept, Avg_age)
SELECT Sdept, AVG(Sage)
FROM Student
GROUP BY Sdept;
```

5.2 修改数据

5.2.1 修改数据的格式

```
1 #语句格式:
2     UPDATE <表名>
3     SET <列名> = <表达式> [ , <列名> = <表达式> ] ...
4     [WHERE <条件>] ;
5
6 #功能:
7 #修改指定表中满足WHERE子句条件的元组
8 #SET子句给出<表达式>的值用于取代相应的属性列
9 #如果省略WHERE子句，表示要修改表中的所有元组
```

修改某一个元组的值

[例3.73]将学生201215121的年龄改为22岁

```
UPDATE Student
SET Sage=22
WHERE Sno = '201215121';
```

修改多个元组的值

[例3.74]将所有学生的年龄增加1岁。

```
UPDATE Student
SET Sage = Sage + 1;
```

5.2.2 带子查询的修改语句

□ [例3.75] 将计算机科学系全体学生的成绩置零。

```
UPDATE SC
SET Grade=0
WHERE Sno IN (
    SELETE Sno
    FROM Student
    WHERE Sdept = 'CS' );
```

5.2.3 修改数据与完整性

关系数据库管理系统在执行修改语句时会检查修改操作是否破坏表上已定义的完整性规划

- 实体完整性
- 参照完整性
- 用户定义的完整性

5.3 删除数据

5.3.1 删除数据的格式

```
1 #语句格式:
2 DELETE
3 FROM <表名>
4 [ WHERE <条件> ] ;
5 #功能:
6 删除指定表中满足WHERE子句条件的元组
7 #WHERE子句:
8 指定要删除的元组
9 缺省表示要删除表中的全部元组, 表的定义仍在字典中
```

删除某一个元组的值

[例3.76] 删除学号为201215128的学生记录。

```
DELETE
FROM Student
WHERE Sno = '201215128';
```

删除多个元组的值

[例3.77] 删除所有的学生选课记录。

```
DELETE
FROM SC;
```

5.3.2 带子查询的删除语句

[例3.78] 删除计算机科学系所有学生的选课记录。

```
DELETE
FROM SC
WHERE Sno IN (
    SELETE Sno
    FROM Student
    WHERE Sdept= 'CS') ;
```

6.空值的处理

空值就是“不知道”或“不存在”或“无意义”的值。SQL允许某些元组的某些属性在一定情况下取空值。

- 该属性应该有一个值，但是目前不知道它的具体值
- 该属性不应该有值
- 由于某种原因不便于填写

6.1 空值的产生

修改某个元组中的某个属性为NULL，外连接会产生空值，空值的关系运算会产生空值。

6.2 空值的判断

用IS NULL或IS NOT NULL判断。

□ [例3.81] 从Student表中找出漏填了数据的学生信息

```
SELECT *
FROM Student
WHERE Sname IS NULL or
      Ssex IS NULL or
      Sage IS NULL or
      Sdept IS NULL;
```

6.3 空值的约束条件

属性定义（或域定义）中有NOT NULL约束条件的不能取空值，加了UNIQUE限制的属性不能取空值，码属性不能取空值。

6.4 空值的算术运算、比较运算和逻辑运算

- 空值与另一个值（包括空值）的算术运算的结果为空值
- 空值与另一个值（包括空值）的比较运算的结果为UNKNOWN
- 有了UNKNOWN后，传统的逻辑运算中的二值逻辑就扩展成了三值逻辑

表 3.8 逻辑运算符真值表

x	y	x AND y	x OR y	NOT x
T	T	T	T	F
T	U	U	T	F
T	F	F	T	F
U	T	U	T	U
U	U	U	U	U
U	F	F	U	U
F	T	F	T	T
F	U	F	U	T
F	F	F	F	T

在查询语句中，只有使WHERE和HAVING子句中的选择条件为TRUE的元组才被选出作为输出的结果。

因此在关系数据库管理系统中，一般不支持三值逻辑，统一规定：空值参与比较运算的结果为逻辑假FALSE。

□ [例3.83] 选出选修1号课程的不及格的学生以及缺考的学生。

□ (解1)

```
SELECT Sno
FROM SC
WHERE Grade < 60 AND
      Cno = '1'

UNION

SELECT Sno
FROM SC
WHERE Grade IS NULL AND
      Cno = '1';
```

□ (解2) 或者

```
SELECT Sno
FROM SC
WHERE Cno = '1' AND
      (Grade < 60 OR
       Grade IS NULL);
```

□ [思考] 能否用下述语句来实现查询3.83?

```
SELECT Sno
FROM SC
WHERE Cno = '1' AND
      NOT (Grade >= 60);
```

7.视图

7.1 特点

- 虚表，是从一个或几个基本表（或视图）导出的表
- 只存放视图的定义，不存放视图对应的数据
- 基表中的数据发生变化，从视图中查询出的数据也随之发生改变

7.2 建立视图

```
1  #语句格式：
2      CREATE VIEW <视图名> [(<列名> [, <列名>]...)]
3      AS <子查询>
4      [WITH CHECK OPTION];
5
6  #WITH CHECK OPTION:
```

7	对视图进行UPDATE, INSERT和DELETE操作时要保证更新、插入或删除的行满足视图定义中的谓词条件（即子查询中的条件表达式）
8	
9	#子查询可以是任意的SELECT语句，是否可以含有ORDER BY子句和DISTINCT短语，取决于具体系统的实现
10	#在标准SQL中，视图对应的子查询中不允许有ORDER BY子句，但可以使用DISTINCT短语
11	
12	#组成视图的属性列名：全部省略 或 全部指定
13	#全部省略：
14	由子查询中SELECT目标列中的诸字段的名字作为视图中对列的列名
15	#明确指定视图的所有列名：
16	具体到每一列，可以继续沿用子查询中对应目标列的列名，也可以启用一个新的更合适的列名，但必须保证视图中列名的唯一性
17	
18	#当出现下列情况时，必须明确指定视图的所有列名：
19	#某个目标列是聚集函数或列表达式
20	#多表连接时选出了几个同名列作为视图的字段
21	#需要在视图为某个列启用新的更合适的名字

关系数据库管理系统执行CREATE VIEW语句时只是把视图定义存入数据字典，并不执行其中的SELECT语句。

在对视图查询时，按视图的定义从基本表中将数据查出。

□ [例3.84] 建立信息系学生的视图。

```
CREATE VIEW IS_Student
AS
SELECT Sno, Sname, Sage
FROM Student
WHERE Sdept = 'IS';
```

- 若一个视图是从单个基本表导出的，并且只是去掉了基本表的某些行和某些列，但保留了主码，这类视图称为行列子集视图

7.3 WITH CHECK OPTION的说明

□ [例3.85] 建立信息系学生的视图，并要求进行修改和插入操作时仍需保证该视图只有信息系的学生。

```
CREATE VIEW IS_Student
AS
SELECT Sno, Sname, Sage
FROM Student
WHERE Sdept = 'IS'
WITH CHECK OPTION;
```

□ 定义IS_Student视图时加上了WITH CHECK OPTION子句，其作用是：如果允许在该视图上执行更新操作，则其更新后的结果元组仍然必需满足视图的定义条件。即通过该视图插入或修改后的新元组能够通过该视图上的查询操作查出来。

□ 视图定义中的WITH CHECK OPTION子句，不影响直接在基本表Student上的元组插入和修改操作的执行。

□例：有如下的信息系学生视图。

```
CREATE VIEW IS_Stud
AS
SELECT Sno,Sname,Sdept
FROM Student
WHERE Sdept = 'IS'
WITH CHECK OPTION;
```

□例：下述元组插入和修改操作都不会得到执行：

```
INSERT INTO IS_Stud
VALUES(231215101,'王海','CS');

UPDATE IS_Stud
SET Sdept = 'CS'
WHERE Sno = '201215121';
```

□但是，在基本表Student上可以执行

```
INSERT
INTO Student(Sno,Sname,Sdept)
VALUES(231215101,'王海','CS');

UPDATE Student
SET Sdept = 'CS'
WHERE Sno = '201215121';
```

7.4 不同的视图类型

基于多个基表的视图

[例3.86] 建立信息系选修了1号课程的学生的视图（包括学号、姓名、成绩）。

```
CREATE VIEW IS_S1(Sno, Sname, Grade)
AS
SELECT Student.Sno, Sname, Grade
FROM Student, SC
WHERE Sdept = 'IS' AND
      Student.Sno = SC.Sno AND
      SC.Cno = '1';
```

基于视图的视图

[例3.87] 建立信息系选修了1号课程且成绩在90分以上的学生的视图。

```
CREATE VIEW IS_S2
AS
SELECT Sno, Sname, Grade
FROM IS_S1
WHERE Grade >= 90;
```

带表达式的视图

[例3.88] 定义一个反映学生出生年份的视图。

```
CREATE VIEW BT_S(Sno, Sname, Sbirth)
AS
SELECT Sno, Sname, 2014 - Sage
FROM Student;
```

分组视图

[例3.89] 将学生的学号及平均成绩定义为一个视图。

```
CREAT VIEW S_G(Sno, Gavg)
AS
SELECT Sno, AVG(Grade)
FROM SC
GROUP BY Sno;
```

7.5 删除视图

```
1 #语句的格式:
2 DROP VIEW <视图名> [CASCADE];
3
4 #该语句从数据字典中删除指定的视图定义
5 #如果该视图上还导出了其他视图, 使用CASCADE级联删除语句, 把该视图和由它导出的所有视图一起删除。否则, 系统将拒绝当前的视图删除操作。
6
7 #在删除基本表时如果使用了CASCADE选项, 由该基本表导出的所有视图定义都会被连带一起删除; 否则, 必须先显式地使用DROP VIEW语句删除所有相关的视图, 然后才能再删除基本表。
```

❑[例3.91] 删除视图BT_S和IS_S1

```
DROP VIEW BT_S;      /*成功执行*/
DROP VIEW IS_S1;     /*拒绝执行*/
```

❑要删除IS_S1, 需使用级联删除:

```
DROP VIEW IS_S1 CASCADE;
```

7.6 视图上的操作

- 查询: 视图上的查询操作将首先被改写为基表上的查询操作, 然后才能得到执行
- 一般不允许执行视图上的更新操作, 只有满足以下条件才可以:
 - 视图的每一行必须对应基表的惟一一行
 - 视图的每一列必须对应基表的惟一列
- 同时满足上述两个条件的视图被称为“可更新视图”
- 视图上的数据更新操作需要被转化为基表上的数据更新操作, 只有在可更新视图上才允许执行数据更新操作

7.7 查询视图

- 用户角度: 查询视图与查询基本表相同
- 关系数据库管理系统实现视图查询的方法:
- 视图消解法:
 - 进行有效性检查
 - 转换成等价的对基本表的查询
 - 执行修正后的查询

- ❑ [例3. 92] 在信息系学生的视图中找出年龄小于20岁的学生。

```
SELECT Sno,Sage
FROM IS_Student
WHERE Sage<20;
```

- ❑ [例3. 93] 查询选修了1号课程的信息系学生

```
SELECT IS_Student.Sno, Sname
FROM IS_Student, SC
WHERE IS_Student.Sno=SC.Sno
AND SC.Cno= '1';
```

- ❑ 视图消解转换后的查询语句为：

```
SELECT Sno,Sage
FROM Student
WHERE Sdept= 'IS' AND
Sage<20;
```

- ❑ [例3. 94] 在S_G视图中查询平均成绩在90分以上的学生学号和平均成绩

```
SELECT *
FROM S_G
WHERE Gavg>=90;
```

- ❑ 视图消解后得到的查询：

```
SELECT Sno,AVG(Grade)
FROM SC
GROUP BY Sno
HAVING AVG(Grade)>=90;
```

- ❑ S_G视图的子查询定义：

```
CREATE VIEW S_G (Sno,Gavg)
AS
SELECT Sno, AVG(Grade)
FROM SC
GROUP BY Sno;
```

- ❑ 或者，直接在基本表上组织查询

```
SELECT *
FROM (SELECT Sno,AVG(Grade)
FROM SC
GROUP BY Sno)
AS S_G(Sno,Gavg)
WHERE Gavg>=90;
```

7.8 更新视图

- ❑ [例3. 95] 将信息系学生视图IS_Student中学号“201215122”的学生姓名改为“刘辰”。

```
UPDATE IS_Student
SET Sname = '刘辰'
WHERE Sno = '201215122';
```

- 转换后的语句：

```
UPDATE Student
SET Sname = '刘辰'
WHERE Sno = '201215122' AND Sdept = 'IS';
```


- ❑[例3.96] 向信息系学生视图IS_S中插入一个新的学生记录，其中学号为“201215129”，姓名为“赵新”，年龄为20岁

```
INSERT  
INTO IS_Student  
VALUES('201215129', '赵新', 20);
```

- ❑转换为对基本表的更新：

```
INSERT  
INTO Student(Sno, Sname, Sage, Sdept)  
VALUES('200215129 ', '赵新', 20, 'IS' );
```

- ❑[例3.97]删除信息系学生视图IS_Student中学号为“201215129”的记录

```
DELETE  
FROM IS_Student  
WHERE Sno = '201215129';
```

- ❑转换为对基本表的更新：

```
DELETE  
FROM Student  
WHERE Sno = '201215129' AND Sdept = 'IS';
```

- 允许对行列子集视图进行更新
- 对其他类型视图的更新不同系统有不同限制，一些视图是不可更新的
- 在一个不允许更新的视图上定义的新视图也不允许更新

7.9 视图的作用

- 视图能够简化用户的操作
- 视图使用户能以多种角度看待同一数据
- 视图对重构数据库提供了一定程度的逻辑独立性

- ❑例：学生关系Student(Sno,Sname,Ssex,Sage,Sdept)

➤“垂直”地分成两个基本表：

SX(Sno, Sname, Sage) 和 SY(Sno, Ssex, Sdept)

- ❑通过建立一个视图Student：

```
CREATE VIEW Student(Sno,Sname,Ssex,Sage,Sdept)  
AS  
SELECT SX.Sno,SX.Sname,SY.Ssex,SX.Sage,SY.Sdept  
FROM SX,SY  
WHERE SX.Sno = SY.Sno;
```

- ❑使用户的外模式保持不变，用户应用程序通过视图仍然能够查找数据
- ❑由于对视图的更新是有条件的，因此应用程序中修改数据的语句可能仍会因基本表结构的改变而改变。

- 视图能够对机密数据提供安全保护
- 适当的利用视图可以更清晰的表达查询

□经常需要执行这样的查询“对每个同学找出他获得最高成绩的课程号”。

□可以先定义一个视图，求出每个同学获得的最高成绩

```
CREATE VIEW VMGRADE
AS
    SELECT Sno, MAX(Grade) Mgrade
    FROM SC
    GROUP BY Sno;
```

□然后用如下的查询语句完成查询：

```
SELECT SC.Sno,Cno
FROM SC, VMGRADE
WHERE SC.Sno=VMGRADE.Sno AND
      SC.Grade=VMGRADE .Mgrade;
```